

Deep Generative Models

Lecture 14

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

2026, Autumn

Recap of Previous Lecture

ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) - D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T)) - \underbrace{\sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{\mathcal{L}_t}$$

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \text{Cat}\left(\frac{\mathbf{Q}_t \mathbf{x}_t \odot \mathbf{Q}_{1:t-1} \mathbf{x}_0}{\mathbf{x}_t^{\top} \mathbf{Q}_{1:t} \mathbf{x}_0}\right).$$

- ▶ Both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $q(\mathbf{x}_t|\mathbf{x}_0)$ are known analytically from the forward process.
- ▶ The reverse process $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is a learned categorical distribution:

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \text{Cat}(\boldsymbol{\pi}_{\theta}(\mathbf{x}_t, t)).$$

Recap of Previous Lecture

ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)).$$

Categorical KL

$$D_{\text{KL}}(\text{Cat}(\mathbf{q}) \parallel \text{Cat}(\mathbf{p})) = \sum_{k=1}^K q_k \log \frac{q_k}{p_k} = H(\mathbf{q}, \mathbf{p}) - H(\mathbf{q}),$$

- ▶ $H(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0))$ is a constant w.r.t. θ .
- ▶ $H(\mathbf{q}, \mathbf{p}) = -\sum_k q_k \log p_k$ is a **cross-entropy loss**.

Therefore, minimizing $\mathcal{L}_t$ w.r.t. θ is equivalent to minimizing

$$\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} H(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0), p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)).$$

Setup: from PF-ODE to few-step samplers

Recall: probability flow ODE

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t) \right] dt, \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

Setup: from PF-ODE to few-step samplers

Recall: probability flow ODE

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t) \right] dt, \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

- Plugging $\mathbf{s}_{\theta}$ into the PF-ODE and using a high-order ODE solver (Heun, RK) reduces the number of steps from 100–1000 down to 20–50.

Setup: from PF-ODE to few-step samplers

Recall: probability flow ODE

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t) \right] dt, \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

- ▶ Plugging $\mathbf{s}_{\theta}$ into the PF-ODE and using a high-order ODEsolve (Heun, RK) reduces the number of steps from 100–1000 down to 20–50.
- ▶ Can we go further — to **1–4** steps?

Setup: from PF-ODE to few-step samplers

Recall: probability flow ODE

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t) \right] dt, \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

- ▶ Plugging $\mathbf{s}_{\theta}$ into the PF-ODE and using a high-order ODEsolve (Heun, RK) reduces the number of steps from 100–1000 down to 20–50.
- ▶ Can we go further — to **1–4** steps?
- ▶ Idea: **learn a shortcut network** that compresses many ODE-solver steps into one.

Setup: from PF-ODE to few-step samplers

Recall: probability flow ODE

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\mathbf{s}(\mathbf{x}, t)] dt, \quad \mathbf{s}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

- ▶ Plugging $\mathbf{s}_\theta$ into the PF-ODE and using a high-order ODE solver (Heun, RK) reduces the number of steps from 100–1000 down to 20–50.
- ▶ Can we go further — to **1–4** steps?
- ▶ Idea: **learn a shortcut network** that compresses many ODE-solver steps into one.

Three approaches in this lecture

- ▶ **Progressive Distillation** — iteratively halve the teacher's step count.
- ▶ **Consistency Models** — enforce self-consistency along PF-ODE trajectories.

Song Y. et al. *Score-Based Generative Modeling through Stochastic Differential Equations* 2020

- ▶ **Distribution Matching Distillation** — match generator and

Outline

1. Progressive Distillation
2. Consistency Models
3. Distribution Matching Distillation

Outline

1. Progressive Distillation
2. Consistency Models
3. Distribution Matching Distillation

Progressive Distillation: motivation

Teacher–student setup

- ▶ Start from a pretrained diffusion teacher $\mathbf{x}_\phi$ with N sampling steps.
- ▶ Train a student $\mathbf{x}_\theta$ that produces the same sample in $N/2$ steps.

Progressive Distillation: motivation

Teacher–student setup

- ▶ Start from a pretrained diffusion teacher $\mathbf{x}_\phi$ with N sampling steps.
- ▶ Train a student $\mathbf{x}_\theta$ that produces the same sample in $N/2$ steps.

Strategy

- ▶ One student step $\equiv$ two teacher steps.
- ▶ Promote the trained student to a new teacher; repeat.

Progressive Distillation: motivation

Teacher–student setup

- ▶ Start from a pretrained diffusion teacher $\mathbf{x}_\phi$ with N sampling steps.
- ▶ Train a student $\mathbf{x}_\theta$ that produces the same sample in $N/2$ steps.

Strategy

- ▶ One student step $\equiv$ two teacher steps.
- ▶ Promote the trained student to a new teacher; repeat.

After K rounds: $N/2^K$ sampling steps (e.g. $1024 \rightarrow 512 \rightarrow \dots \rightarrow 1$).

One distillation step

Two teacher DDIM steps

$$\mathbf{x}_t \xrightarrow{\text{teacher}} \mathbf{x}_{t-\Delta/2} \xrightarrow{\text{teacher}} \mathbf{x}_{t-\Delta}.$$

One distillation step

Two teacher DDIM steps

$$\mathbf{x}_t \xrightarrow{\text{teacher}} \mathbf{x}_{t-\Delta/2} \xrightarrow{\text{teacher}} \mathbf{x}_{t-\Delta}.$$

Student target

Define $\tilde{\mathbf{x}}_0$ such that *one* student DDIM step from $\mathbf{x}_t$ lands on $\mathbf{x}_{t-\Delta}$ (the teacher's two-step endpoint).

One distillation step

Two teacher DDIM steps

$$\mathbf{x}_t \xrightarrow{\text{teacher}} \mathbf{x}_{t-\Delta/2} \xrightarrow{\text{teacher}} \mathbf{x}_{t-\Delta}.$$

Student target

Define $\tilde{\mathbf{x}}_0$ such that *one* student DDIM step from $\mathbf{x}_t$ lands on $\mathbf{x}_{t-\Delta}$ (the teacher's two-step endpoint).

Distillation loss

$$\mathcal{L}_{\text{PD}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[w(t) \|\mathbf{x}_\theta(\mathbf{x}_t, t) - \tilde{\mathbf{x}}_0\|^2 \right].$$

The $\mathbf{v}$ -parameterization

Definition

$$\mathbf{v} := \alpha_t \boldsymbol{\epsilon} - \sigma_t \mathbf{x}_0, \quad \mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}.$$

The $\mathbf{v}$ -parameterization

Definition

$$\mathbf{v} := \alpha_t \boldsymbol{\epsilon} - \sigma_t \mathbf{x}_0, \quad \mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}.$$

- ▶ At low SNR (large t): $\boldsymbol{\epsilon}$ -prediction is uninformative ($\mathbf{x}_t \approx \sigma_t \boldsymbol{\epsilon}$).
- ▶ At high SNR (small t): $\mathbf{x}_0$ -prediction is uninformative ($\mathbf{x}_t \approx \alpha_t \mathbf{x}_0$).

The $\mathbf{v}$ -parameterization

Definition

$$\mathbf{v} := \alpha_t \boldsymbol{\epsilon} - \sigma_t \mathbf{x}_0, \quad \mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}.$$

- ▶ At low SNR (large t): $\boldsymbol{\epsilon}$ -prediction is uninformative ($\mathbf{x}_t \approx \sigma_t \boldsymbol{\epsilon}$).
- ▶ At high SNR (small t): $\mathbf{x}_0$ -prediction is uninformative ($\mathbf{x}_t \approx \alpha_t \mathbf{x}_0$).
- ▶ $\mathbf{v}$ -prediction stays well-conditioned across the whole schedule.

The $\mathbf{v}$ -parameterization

Definition

$$\mathbf{v} := \alpha_t \boldsymbol{\epsilon} - \sigma_t \mathbf{x}_0, \quad \mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}.$$

- ▶ At low SNR (large t): $\boldsymbol{\epsilon}$ -prediction is uninformative ($\mathbf{x}_t \approx \sigma_t \boldsymbol{\epsilon}$).
- ▶ At high SNR (small t): $\mathbf{x}_0$ -prediction is uninformative ($\mathbf{x}_t \approx \alpha_t \mathbf{x}_0$).
- ▶ $\mathbf{v}$ -prediction stays well-conditioned across the whole schedule.

Conversion identities

$$\mathbf{x}_0 = \alpha_t \mathbf{x}_t - \sigma_t \mathbf{v}, \quad \boldsymbol{\epsilon} = \sigma_t \mathbf{x}_t + \alpha_t \mathbf{v}.$$

The $\mathbf{v}$ -parameterization

Definition

$$\mathbf{v} := \alpha_t \boldsymbol{\epsilon} - \sigma_t \mathbf{x}_0, \quad \mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}.$$

- ▶ At low SNR (large t): $\boldsymbol{\epsilon}$ -prediction is uninformative ($\mathbf{x}_t \approx \sigma_t \boldsymbol{\epsilon}$).
- ▶ At high SNR (small t): $\mathbf{x}_0$ -prediction is uninformative ($\mathbf{x}_t \approx \alpha_t \mathbf{x}_0$).
- ▶ $\mathbf{v}$ -prediction stays well-conditioned across the whole schedule.

Conversion identities

$$\mathbf{x}_0 = \alpha_t \mathbf{x}_t - \sigma_t \mathbf{v}, \quad \boldsymbol{\epsilon} = \sigma_t \mathbf{x}_t + \alpha_t \mathbf{v}.$$

Progressive (iterative) halving

Algorithm

1. Initialize student $\theta \leftarrow \phi$ (copy of teacher).
2. For $k = 1, \dots, K$:
 - ▶ Train $\mathbf{x}_\theta$ to match *two* steps of the current teacher with *one* step.
 - ▶ Promote: $\phi \leftarrow \theta$; halve the sampling schedule.

Progressive (iterative) halving

Algorithm

1. Initialize student $\theta \leftarrow \phi$ (copy of teacher).
2. For $k = 1, \dots, K$:
 - ▶ Train x_θ to match *two* steps of the current teacher with *one* step.
 - ▶ Promote: $\phi \leftarrow \theta$; halve the sampling schedule.
- ▶ Each round uses the previous student as its teacher — no retraining from scratch.
- ▶ Final inference cost: $N/2^K$ steps, often 1–4.

Loss weighting and limitations

Truncated-SNR weighting

$$w(t) = \max(\text{SNR}(t), 1), \quad \text{SNR}(t) = \alpha_t^2 / \sigma_t^2.$$

Prevents the high-SNR end of the schedule from dominating as the student approaches 1-step inference.

Loss weighting and limitations

Truncated-SNR weighting

$$w(t) = \max(\text{SNR}(t), 1), \quad \text{SNR}(t) = \alpha_t^2 / \sigma_t^2.$$

Prevents the high-SNR end of the schedule from dominating as the student approaches 1-step inference.

Limitation

Errors accumulate across rounds — each student inherits the imperfections of its teacher.

⇒ Motivates a *trajectory-level* consistency objective (next section).

Outline

1. Progressive Distillation
2. Consistency Models
3. Distribution Matching Distillation

Consistency function

Definition

A consistency function $f_{\theta} : (\mathbf{x}_t, t) \mapsto \mathbf{x}_{\epsilon}$ maps any point on a PF-ODE trajectory to its origin (data side).

Consistency function

Definition

A consistency function $f_\theta : (\mathbf{x}_t, t) \mapsto \mathbf{x}_\epsilon$ maps any point on a PF-ODE trajectory to its origin (data side).

Self-consistency property

$$f_\theta(\mathbf{x}_t, t) = f_\theta(\mathbf{x}_{t'}, t'), \quad \forall t, t' \in [\epsilon, T],$$

whenever $\mathbf{x}_t$ and $\mathbf{x}_{t'}$ lie on the same PF-ODE trajectory.

Consistency function

Definition

A consistency function $f_\theta : (\mathbf{x}_t, t) \mapsto \mathbf{x}_\epsilon$ maps any point on a PF-ODE trajectory to its origin (data side).

Self-consistency property

$$f_\theta(\mathbf{x}_t, t) = f_\theta(\mathbf{x}_{t'}, t'), \quad \forall t, t' \in [\epsilon, T],$$

whenever $\mathbf{x}_t$ and $\mathbf{x}_{t'}$ lie on the same PF-ODE trajectory.

This is the *defining* property — the entire framework is built by enforcing it.

Boundary condition and parameterization

Boundary condition

$$f_{\theta}(\mathbf{x}_{\epsilon}, \epsilon) = \mathbf{x}_{\epsilon}.$$

Boundary condition and parameterization

Boundary condition

$$f_{\theta}(\mathbf{x}_{\epsilon}, \epsilon) = \mathbf{x}_{\epsilon}.$$

Skip-connection parameterization

$$f_{\theta}(\mathbf{x}, t) = c_{\text{skip}}(t) \mathbf{x} + c_{\text{out}}(t) F_{\theta}(\mathbf{x}, t),$$

with $c_{\text{skip}}(\epsilon) = 1$ and $c_{\text{out}}(\epsilon) = 0$.

Boundary condition and parameterization

Boundary condition

$$f_{\theta}(\mathbf{x}_{\epsilon}, \epsilon) = \mathbf{x}_{\epsilon}.$$

Skip-connection parameterization

$$f_{\theta}(\mathbf{x}, t) = c_{\text{skip}}(t) \mathbf{x} + c_{\text{out}}(t) F_{\theta}(\mathbf{x}, t),$$

with $c_{\text{skip}}(\epsilon) = 1$ and $c_{\text{out}}(\epsilon) = 0$.

- ▶ The boundary condition is enforced *structurally*, not by the loss.
- ▶ Optimization only needs to learn self-consistency.

Consistency Distillation (CD)

Assume a pretrained teacher score $\mathbf{s}_\phi$. Discretize $[\epsilon, T]$ into $\{t_1, \dots, t_N\}$.

Consistency Distillation (CD)

Assume a pretrained teacher score $\mathbf{s}_\phi$. Discretize $[\epsilon, T]$ into $\{t_1, \dots, t_N\}$.

One ODE-solver step (using teacher)

$$\hat{\mathbf{x}}_{t_n} = \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1}) \Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \phi).$$

Consistency Distillation (CD)

Assume a pretrained teacher score $\mathbf{s}_\phi$. Discretize $[\epsilon, T]$ into $\{t_1, \dots, t_N\}$.

One ODE-solver step (using teacher)

$$\hat{\mathbf{x}}_{t_n} = \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1}) \Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \phi).$$

CD loss

$$\mathcal{L}_{\text{CD}}(\theta) = \mathbb{E} \left[\lambda(t_n) d(f_\theta(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\theta^-}(\hat{\mathbf{x}}_{t_n}, t_n)) \right],$$

where θ^- is the EMA of θ (stop-gradient target).

Consistency Distillation (CD)

Assume a pretrained teacher score $\mathbf{s}_\phi$. Discretize $[\epsilon, T]$ into $\{t_1, \dots, t_N\}$.

One ODE-solver step (using teacher)

$$\hat{\mathbf{x}}_{t_n} = \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1}) \Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \phi).$$

CD loss

$$\mathcal{L}_{\text{CD}}(\theta) = \mathbb{E} \left[\lambda(t_n) d(f_\theta(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\theta^-}(\hat{\mathbf{x}}_{t_n}, t_n)) \right],$$

where θ^- is the EMA of θ (stop-gradient target).

The EMA target prevents trivial collapse to a constant function.

Consistency Training (CT)

Replace the teacher's ODE step with the noise-injection identity from denoising score matching.

Consistency Training (CT)

Replace the teacher's ODE step with the noise-injection identity from denoising score matching.

Pair construction (teacher-free)

$$\mathbf{x}_{t_{n+1}} = \mathbf{x}_0 + t_{n+1} \epsilon, \quad \hat{\mathbf{x}}_{t_n} = \mathbf{x}_0 + t_n \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}).$$

Same $\mathbf{x}_0$ and same ϵ for both points.

Consistency Training (CT)

Replace the teacher's ODE step with the noise-injection identity from denoising score matching.

Pair construction (teacher-free)

$$\mathbf{x}_{t_{n+1}} = \mathbf{x}_0 + t_{n+1} \epsilon, \quad \hat{\mathbf{x}}_{t_n} = \mathbf{x}_0 + t_n \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}).$$

Same $\mathbf{x}_0$ and same ϵ for both points.

CT loss

$$\mathcal{L}_{\text{CT}}(\theta) = \mathbb{E} \left[\lambda(t_n) d(f_{\theta}(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\theta}(\hat{\mathbf{x}}_{t_n}, t_n)) \right].$$

Consistency Training (CT)

Replace the teacher's ODE step with the noise-injection identity from denoising score matching.

Pair construction (teacher-free)

$$\mathbf{x}_{t_{n+1}} = \mathbf{x}_0 + t_{n+1} \epsilon, \quad \hat{\mathbf{x}}_{t_n} = \mathbf{x}_0 + t_n \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}).$$

Same $\mathbf{x}_0$ and same ϵ for both points.

CT loss

$$\mathcal{L}_{\text{CT}}(\theta) = \mathbb{E} \left[\lambda(t_n) d(f_{\theta}(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\theta}(\hat{\mathbf{x}}_{t_n}, t_n)) \right].$$

Claim: $\nabla_{\theta} \mathcal{L}_{\text{CT}} = \nabla_{\theta} \mathcal{L}_{\text{CD}} + o(\Delta t)$ as $N \rightarrow \infty$.

Sampling from a consistency model

Single-step sampling

$$\mathbf{x}_T \sim \mathcal{N}(0, T^2 \mathbf{I}), \quad \mathbf{x}_0 \approx f_{\theta}(\mathbf{x}_T, T).$$

Sampling from a consistency model

Single-step sampling

$$\mathbf{x}_T \sim \mathcal{N}(0, T^2 \mathbf{I}), \quad \mathbf{x}_0 \approx f_{\theta}(\mathbf{x}_T, T).$$

Multi-step refinement

Alternate (for $\tau_1 > \tau_2 > \dots > \tau_M$):

1. Denoise: $\mathbf{x}_0 \leftarrow f_{\theta}(\mathbf{x}_{\tau_i}, \tau_i)$.
2. Re-noise: $\mathbf{x}_{\tau_{i+1}} \leftarrow \mathbf{x}_0 + \tau_{i+1} \epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$.

Sampling from a consistency model

Single-step sampling

$$\mathbf{x}_T \sim \mathcal{N}(0, T^2 \mathbf{I}), \quad \mathbf{x}_0 \approx f_{\theta}(\mathbf{x}_T, T).$$

Multi-step refinement

Alternate (for $\tau_1 > \tau_2 > \dots > \tau_M$):

1. Denoise: $\mathbf{x}_0 \leftarrow f_{\theta}(\mathbf{x}_{\tau_i}, \tau_i)$.
2. Re-noise: $\mathbf{x}_{\tau_{i+1}} \leftarrow \mathbf{x}_0 + \tau_{i+1} \epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$.

Interpretation: iterative projection onto the consistency manifold trades a few extra NFEs for higher quality.

Latent Consistency Models (LCM)

Apply Consistency Distillation to a latent diffusion model (Stable Diffusion).

Latent Consistency Models (LCM)

Apply Consistency Distillation to a latent diffusion model (Stable Diffusion).

Guided consistency distillation

Absorb classifier-free guidance into f_θ : an augmented function $f_\theta(\mathbf{x}_t, t, \omega, \mathbf{c})$ takes the guidance scale ω as input. The teacher uses CFG:

$$\tilde{\mathbf{s}}_\phi(\mathbf{x}_t, t, \omega, \mathbf{c}) = (1 + \omega) \mathbf{s}_\phi(\mathbf{x}_t, t, \mathbf{c}) - \omega \mathbf{s}_\phi(\mathbf{x}_t, t, \emptyset).$$

Latent Consistency Models (LCM)

Apply Consistency Distillation to a latent diffusion model (Stable Diffusion).

Guided consistency distillation

Absorb classifier-free guidance into f_{θ} : an augmented function $f_{\theta}(\mathbf{x}_t, t, \omega, \mathbf{c})$ takes the guidance scale ω as input. The teacher uses CFG:

$$\tilde{\mathbf{s}}_{\phi}(\mathbf{x}_t, t, \omega, \mathbf{c}) = (1 + \omega) \mathbf{s}_{\phi}(\mathbf{x}_t, t, \mathbf{c}) - \omega \mathbf{s}_{\phi}(\mathbf{x}_t, t, \emptyset).$$

Skipping-step

Enforce consistency over k steps instead of 1:

$$f_{\theta}(\mathbf{x}_{t_{n+k}}, t_{n+k}) \approx f_{\theta}(\hat{\mathbf{x}}_{t_n}, t_n).$$

Larger SNR gap $\Rightarrow$ stronger training signal.

Outline

1. Progressive Distillation
2. Consistency Models
3. Distribution Matching Distillation

One-step generator

Setup

Train a generator $G_\theta : \mathbf{z} \mapsto \mathbf{x}$, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

One-step generator

Setup

Train a generator $G_{\theta} : \mathbf{z} \mapsto \mathbf{x}$, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Objective

$$G_{\theta\#}p(\mathbf{z}) \approx p_{\text{data}}(\mathbf{x}).$$

Distribution matching, not per-sample matching.

One-step generator

Setup

Train a generator $G_\theta : \mathbf{z} \mapsto \mathbf{x}$, $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$.

Objective

$$G_\theta \# p(\mathbf{z}) \approx p_{\text{data}}(\mathbf{x}).$$

Distribution matching, not per-sample matching.

- ▶ PD and CM tie each $\mathbf{z}$ to a specific PF-ODE endpoint.
- ▶ DMD lets G_θ re-organize the noise-to-image map — $\mathbf{z}$ and $G_\theta(\mathbf{z})$ need not lie on a teacher trajectory.

KL gradient via score difference

Diffuse the generator output: $\mathbf{x}_t = G_\theta(\mathbf{z}) + \sigma_t \epsilon$. Let $p_{\text{fake},t}$ and $p_{\text{real},t}$ be the corresponding marginals.

KL gradient via score difference

Diffuse the generator output: $\mathbf{x}_t = G_\theta(\mathbf{z}) + \sigma_t \epsilon$. Let $p_{\text{fake},t}$ and $p_{\text{real},t}$ be the corresponding marginals.

Variational score-distillation identity

$$\nabla_{\theta} D_{\text{KL}}(p_{\text{fake},t} \parallel p_{\text{real},t}) = -\mathbb{E} \left[(\mathbf{s}_{\text{real},t}(\mathbf{x}_t) - \mathbf{s}_{\text{fake},t}(\mathbf{x}_t)) \cdot \nabla_{\theta} G_{\theta}(\mathbf{z}) \right].$$

KL gradient via score difference

Diffuse the generator output: $\mathbf{x}_t = G_\theta(\mathbf{z}) + \sigma_t \epsilon$. Let $p_{\text{fake},t}$ and $p_{\text{real},t}$ be the corresponding marginals.

Variational score-distillation identity

$$\nabla_{\theta} D_{\text{KL}}(p_{\text{fake},t} \parallel p_{\text{real},t}) = -\mathbb{E} \left[(\mathbf{s}_{\text{real},t}(\mathbf{x}_t) - \mathbf{s}_{\text{fake},t}(\mathbf{x}_t)) \cdot \nabla_{\theta} G_{\theta}(\mathbf{z}) \right].$$

- ▶ Likelihood-free: only score *differences* matter.
- ▶ Integrate over t with weighting $w(t) \Rightarrow$ tractable training signal.

Two score networks

Real score (frozen)

$\mathbf{s}_{\text{real},t} = \mathbf{s}_{\phi}(\mathbf{x}_t, t)$: the pretrained teacher.

Two score networks

Real score (frozen)

$\mathbf{s}_{\text{real},t} = \mathbf{s}_{\phi}(\mathbf{x}_t, t)$: the pretrained teacher.

Fake score (online)

$\mathbf{s}_{\text{fake},t} = \mathbf{s}_{\psi}(\mathbf{x}_t, t)$: a *second* score network trained online with denoising score matching on samples from the current G_{θ} .

Tracks p_{fake} as the generator evolves.

Two score networks

Real score (frozen)

$\mathbf{s}_{\text{real},t} = \mathbf{s}_{\phi}(\mathbf{x}_t, t)$: the pretrained teacher.

Fake score (online)

$\mathbf{s}_{\text{fake},t} = \mathbf{s}_{\psi}(\mathbf{x}_t, t)$: a *second* score network trained online with denoising score matching on samples from the current G_{θ} .

Tracks p_{fake} as the generator evolves.

Alternating optimization

1. Update $\mathbf{s}_{\psi}$ on a batch from G_{θ} (score matching).
2. Update G_{θ} with the score-difference gradient.

DMD1: total loss

Total objective

$$\mathcal{L}_{\text{DMD}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{KL-grad}}(\boldsymbol{\theta}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}(\boldsymbol{\theta}).$$

DMD1: total loss

Total objective

$$\mathcal{L}_{\text{DMD}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{KL-grad}}(\boldsymbol{\theta}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}(\boldsymbol{\theta}).$$

Regression term

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{z}, \mathbf{x}^*)} \|G_{\boldsymbol{\theta}}(\mathbf{z}) - \mathbf{x}^*\|^2,$$

on a few hundred precomputed $(\mathbf{z}, \mathbf{x}^*)$ pairs ($\mathbf{x}^*$ = teacher's PF-ODE output for $\mathbf{z}$).

DMD1: total loss

Total objective

$$\mathcal{L}_{\text{DMD}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{KL-grad}}(\boldsymbol{\theta}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}(\boldsymbol{\theta}).$$

Regression term

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{z}, \mathbf{x}^*)} \|G_{\boldsymbol{\theta}}(\mathbf{z}) - \mathbf{x}^*\|^2,$$

on a few hundred precomputed $(\mathbf{z}, \mathbf{x}^*)$ pairs ($\mathbf{x}^*$ = teacher's PF-ODE output for $\mathbf{z}$).

Interpretation: regression anchors $G_{\boldsymbol{\theta}}$ near the teacher's deterministic map; the KL-gradient term then re-shapes the *distribution* of outputs.

DMD2: improvements

Three changes over DMD1:

DMD2: improvements

Three changes over DMD1:

- ▶ **GAN loss:** a discriminator on real images corrects the bias from an imperfect $\mathbf{s}_{\psi}$.

DMD2: improvements

Three changes over DMD1:

- ▶ **GAN loss**: a discriminator on real images corrects the bias from an imperfect $\mathbf{s}_{\psi}$.
- ▶ **No regression**: removes the need for precomputed $(\mathbf{z}, \mathbf{x}^*)$ pairs; simpler pipeline.

DMD2: improvements

Three changes over DMD1:

- ▶ **GAN loss:** a discriminator on real images corrects the bias from an imperfect $\mathbf{s}_{\psi}$.
- ▶ **No regression:** removes the need for precomputed $(\mathbf{z}, \mathbf{x}^*)$ pairs; simpler pipeline.
- ▶ **Multi-step generator:** G_{θ} becomes a 4-step network, recovering quality lost at the strict 1-step setting.

DMD2: improvements

Three changes over DMD1:

- ▶ **GAN loss:** a discriminator on real images corrects the bias from an imperfect $\mathbf{s}_{\psi}$.
- ▶ **No regression:** removes the need for precomputed $(\mathbf{z}, \mathbf{x}^*)$ pairs; simpler pipeline.
- ▶ **Multi-step generator:** G_{θ} becomes a 4-step network, recovering quality lost at the strict 1-step setting.

Closes most of the quality gap between distilled and full-step diffusion.

Summary

- ▶ PF-ODE solvers (Heun, RK) floor at ~ 20 steps; further acceleration requires **learning a shortcut network**.
- ▶ **Progressive Distillation**: iteratively halve teacher steps via two-step matching, with **v**-parameterization for SNR-uniform optimization.
- ▶ **Consistency Models**: enforce *self-consistency* along PF-ODE trajectories; the boundary condition makes one-step sampling fall out of the parameterization. LCM extends CD to latent SD with guided distillation and skipping-step.
- ▶ **DMD**: match generator and data *distributions* via the score-difference gradient; two score networks (frozen real, online fake) in alternating optimization. DMD2 adds GAN, drops regression, adds multi-step.
- ▶ Tradeoffs: PD is simplest, CM is most principled, DMD currently delivers the best 1-step quality.